TravelSite CI/CD with Jenkins, Docker, AWS ECR, and ECS

Overview

This project demonstrates how to set up a complete CI/CD pipeline using Jenkins, Docker, AWS ECR, and ECS to deploy a static travel website (index.html). The pipeline automatically:

1. Clones the code from GitHub
2. Runs a Build stage (placeholder for future compilation steps)
3. Builds a Docker image
4. Tests the container locally
5. Pushes the image to Amazon Elastic Container Registry (ECR)
6. Deploys the new version to Amazon Elastic Container Service (ECS)

🛠️ Prerequisites

1. AWS Setup
    - AWS Account with IAM User having permissions for ECS, ECR, and IAM.
    - AWS CLI installed and configured on the Jenkins server.

ECR Repository

```
aws ecr create-repository --repository-name firstrepo --region ap-south-1
```

2.

ECS Cluster & Service

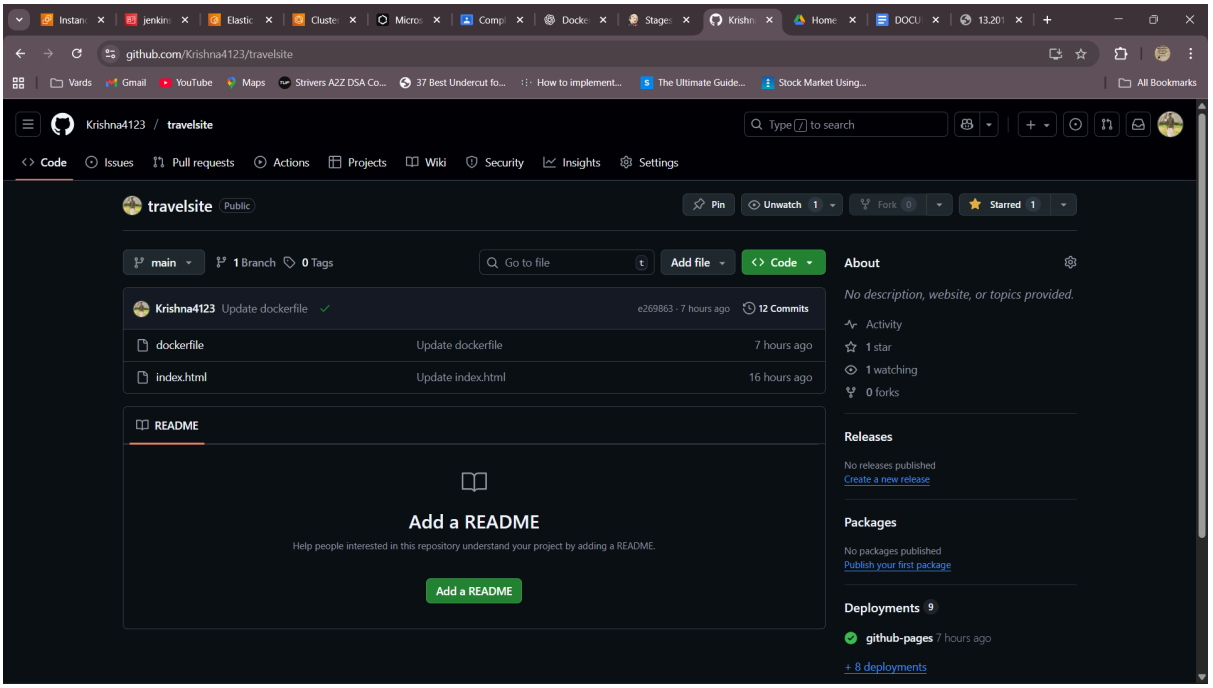
```
aws ecs create-cluster --cluster-name travelsite-cluster
aws ecs create-service \
  --cluster travelsite-cluster \
  --service-name travelsite-service \
  --task-definition travelsite-task \
  --desired-count 1
```

3.
4. Jenkins Server
    - Installed on EC2 (Ubuntu).
    - Plugins: Pipeline, AWS Credentials, Git.

- Docker installed and configured for Jenkins user.

5. GitHub Repository

   - Code is hosted in: [Krishna4123/travelsite](https://github.com/Krishna4123/travelsite).



📂 Project Structure

```
travelsite/
├── index.html
├── Dockerfile
└── Jenkinsfile
```

- index.html → Static Travel Website page.
- Dockerfile → Builds an Nginx image serving index.html.
- Jenkinsfile → Pipeline definition.

🐳 Dockerfile

```
FROM nginx:stable-alpine
COPY index.html /usr/share/nginx/html/
EXPOSE 80
```

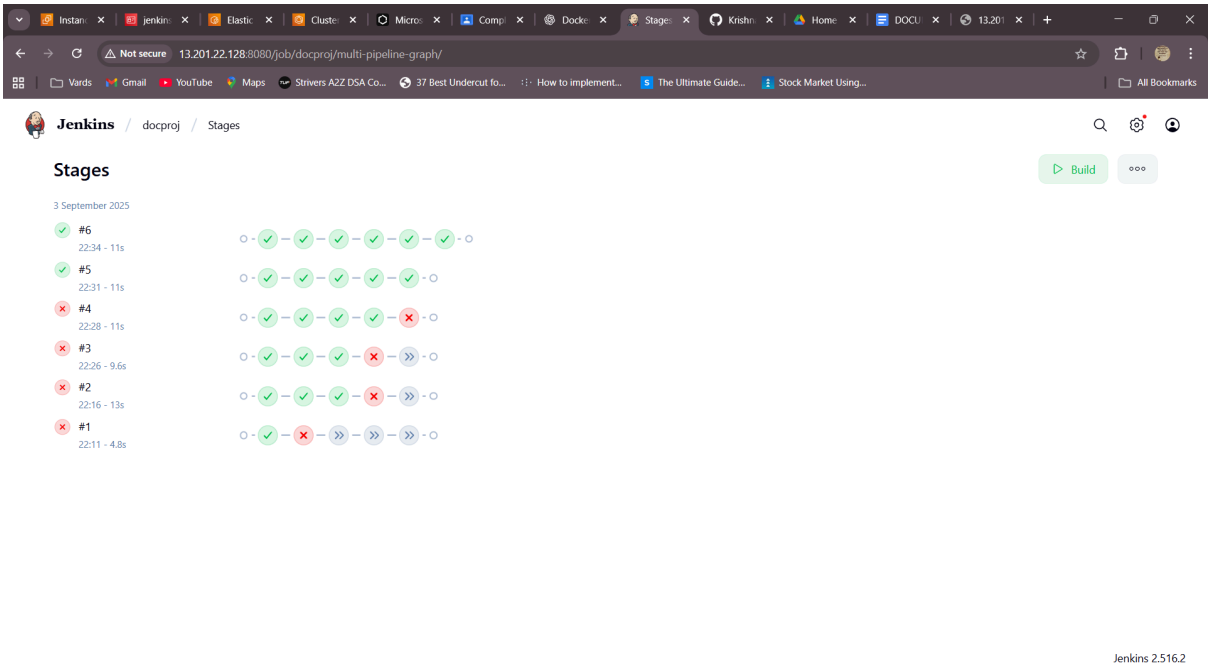
⚙️ Jenkins Pipeline (Jenkinsfile)

The pipeline defines stages: Clone → Build → Build Docker Image → Test → Push → Deploy.

```
pipeline {
```

```
    agent any

    environment {
        APP_NAME     = "travelsite"
        IMAGE_NAME   = "travelsite:test"
        AWS_REGION   = "ap-south-1"
        ECR_REPO     = "538712952108.dkr.ecr.ap-south-1.amazonaws.com/firstrepo"
        ECS_CLUSTER  = "travelsite-cluster"
        ECS_SERVICE  = "travelsite-service"
    }

    stages {
        stage('Clone') {
            steps {
                sh "rm -rf ${APP_NAME} && git clone
https://github.com/Krishna4123/travelsite.git"
            }
        }

        stage('Build') {
            steps {
                dir(APP_NAME) {
                    sh "echo 'Running build step... (currently just static files)'"
                }
            }
        }

        stage('Build Docker Image') {
            steps {
                dir(APP_NAME) {
                    sh "docker build -t ${IMAGE_NAME} ."
                }
            }
        }

        stage('Test Docker Container') {
            steps {
                script {
                    sh 'docker rm -f test-container || true'
                    sh 'docker run -d -p 8090:80 --name test-container travelsite:test'
                    sh 'sleep 5'
                    sh 'curl -f http://localhost:8090/index.html'
                    sh 'docker rm -f test-container'
                }
            }
        }

        stage('Push to ECR') {
```

```
        steps {
            withCredentials([usernamePassword(credentialsId: 'aws-jenkins',
usernameVariable: 'AWS_ACCESS_KEY_ID', passwordVariable:
'AWS_SECRET_ACCESS_KEY')]) {
                script {
                    def IMAGE_TAG = "latest"
                    def ECR_URI  = "${ECR_REPO}"

                    sh """
                    export AWS_ACCESS_KEY_ID=$AWS_ACCESS_KEY_ID
                    export AWS_SECRET_ACCESS_KEY=$AWS_SECRET_ACCESS_KEY
                    export AWS_DEFAULT_REGION=${AWS_REGION}

                    aws ecr get-login-password --region ${AWS_REGION} \
                    | docker login --username AWS --password-stdin $ECR_URI

                    docker tag ${IMAGE_NAME} $ECR_URI:$IMAGE_TAG
                    docker push $ECR_URI:$IMAGE_TAG
                    """
                }
            }
        }
    }

    stage('Deploy to ECS') {
        steps {
            withCredentials([usernamePassword(credentialsId: 'aws-jenkins',
usernameVariable: 'AWS_ACCESS_KEY_ID', passwordVariable:
'AWS_SECRET_ACCESS_KEY')]) {
                script {
                    sh """
                    export AWS_ACCESS_KEY_ID=$AWS_ACCESS_KEY_ID
                    export AWS_SECRET_ACCESS_KEY=$AWS_SECRET_ACCESS_KEY
                    export AWS_DEFAULT_REGION=${AWS_REGION}

                    aws ecs update-service \
                      --cluster ${ECS_CLUSTER} \
                      --service ${ECS_SERVICE} \
                      --force-new-deployment \
                      --region ${AWS_REGION}
                    """
                }
            }
        }
    }
  }
}
```



🔄 Pipeline Workflow

1. Clone → Pulls latest code from GitHub.
2. Build → Placeholder (can be used for React, Angular, Java builds).
3. Build Docker Image → Creates Nginx image with app code.
4. Test Docker Container → Starts container, verifies with curl.
5. Push to ECR → Tags and uploads image to AWS ECR.
6. Deploy to ECS → Forces ECS service to use the new image.

ap-south-1.console.aws.amazon.com/ecr/repositories/private/538712952108/firstrepo/_/image/sha256:0fdd14cb1107ad1e0ee626bb8795fd0c4435b333acd3c77646019ae0ab161729/details?re...

Account ID: 5387-1295-2108 _Krishna04

Amazon ECR > Private registry > Repositories > firstrepo > sha256:0fdd14cb1107ad1e0ee626bb8795fd0c4435b333acd3c77646019ae0ab161729

Amazon Elastic Container Registry

- Private registry
  - Repositories
    - Summary
    - Images
    - Permissions
    - Lifecycle Policy
    - Repository tags
  - Features & Settings
- Public registry
  - Repositories
  - Settings

ECR public gallery
Amazon ECS
Amazon EKS

Getting started
Documentation

Image

Details

Image tags
latest

URI
538712952108.dkr.ecr.ap-south-1.amazonaws.com/firstrepo:latest

Digest
sha256:0fdd14cb1107ad1e0ee626bb8795fd0c4435b333acd3c77646019ae0ab161729

General information

Artifact type	Repository	Pushed at
Image	firstrepo	September 03, 2025, 21:40:12 (UTC+05.5)
Last recorded pull time	**Size (MB)**	
-	20.96	

Scanning and vulnerabilities

Scan

Status
Scan not found

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

ap-south-1.console.aws.amazon.com/ecs/v2/task-definitions/travelsite-task/1/containers?region=ap-south-1

Account ID: 5387-1295-2108 _Krishna04

Amazon Elastic Container Service > Task definitions > travelsite-task > Revision 1 > Containers

Amazon Elastic Container Service

- Clusters
- Namespaces
- Task definitions
- Account settings

Amazon ECR
Repositories

AWS Batch

Documentation
Discover products
Subscriptions

Tell us what you think

travelsite-task:1

Last updated September 3, 2025 at 22:40 (UTC+5:30)

Deploy | Actions | Create new revision

Overview Info

ARN	Status	Time created	App environment
arn:aws:ecs:ap-south-1:538712952108:task-definition/travelsite-task:1	ACTIVE	September 3, 2025 at 22:05 (UTC+5:30)	Fargate
Task role	**Task execution role**	**Operating system/Architecture**	**Network mode**
-	ecsTaskExecutionRole	Linux/X86_64	awsvpc

Fault injection
Turned off

Containers | JSON | Task placement | Volumes (0) | Requires attributes | Tags

Task size

Task CPU
1,024 units (1 vCPU)

Task CPU maximum allocation for containers

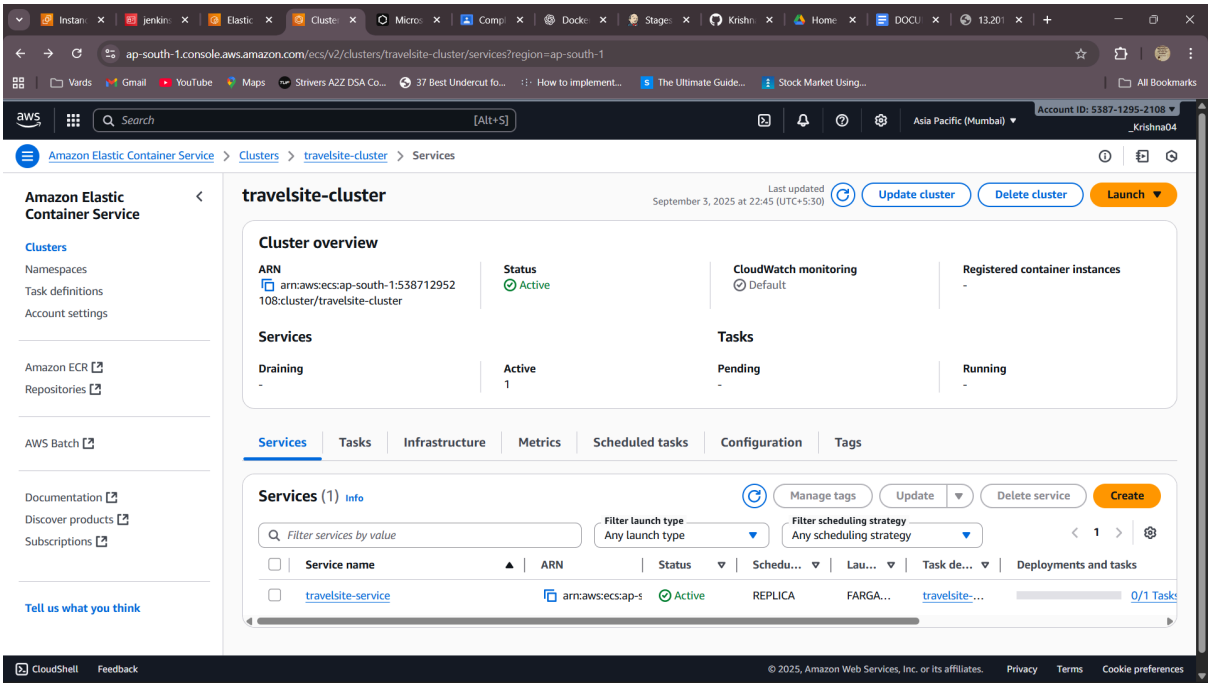
CPU (unit) 0 100 200 300 400 500 600 700 800 900 1000

Task memory
3,072 MiB (3 GB)

Task memory maximum allocation for container memory reservation

Memory (MiB) 0 500 1000 1500 2000 2500 3000

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences



✅ Verification

After successful deployment:

- Go to AWS ECS Console → Check travelsite-service.

- Find the Public IP of ECS task → Open http://<ECS-IP>/index.html.